Design Decision: How the Agent Represents a Student's Mental Model

Status: **Accepted for the class proof of concept (2026-11-03)**. A restructuring is planned (§9). Owner (design authority): Suzanne Szabo Related: `docs/decisions.md` D2 and D10, `mm_agent/catalog.py`, `mm_agent/probes.py`, `mm_agent/belief.py`

This document records the most important design choice in the project: what data structure stands for "the student's mental model." Everything else (the probes, the belief update, probe selection, the LLM interpreter, the evaluation and the hand-off to a lesson engine) is built on this choice. If the representation is wrong, no amount of good agent design downstream can recover.

---

1. Summary

The short version: we represent a student's mental model as *the correct notional machine with some of its rules replaced by known wrong versions*. Concretely, a mental model is a subset of a fixed catalog of eight misconceptions. The empty subset is a viable model. The agent never commits to one answer; it keeps a probability for every possible subset and narrows them down as the student answers probes.

Five points that explain it:

1. **A notional machine is a small set of rules** for how code executes (assignment binds a name; `def` does not run the body; a call creates a frame; locals vanish on return; `return` ends the call).
2. **Each misconception is a wrong version of one rule**, taken from the CS-education research literature.
3. **A student's mental model = the correct rules with some swapped out.** That makes it *executable*: for any program, we can predict what this student will say it prints.
4. **Prediction is what makes inference possible.** Comparing what each candidate model predicts with what the student actually answers tells the agent which model is most likely. That is an ordinary Bayesian update.
5. **This idea is proven.** Brown and Burton's BUGGY (1978) diagnosed children's subtraction errors the same way, as a correct procedure with specific "bugs" substituted in. We apply the idea to the notional machine of Python function calls.

Why this representation rather than another: it is the only option considered that is at once *measurable* (can be scored against ground truth), *predictive* (says what the student will answer), *actionable* (each entry tells a lesson what to fix), *interpretable*, and *buildable without student data*. See the comparison in §6.

Main limitation: it can only diagnose misconceptions that are already in the catalog (§8).

---

2. The problem: what the representation must do

The project asks whether an agent can reasonably and accurately determine where a student is in their understanding of a lesson's notional machine before a dynamic lesson begins. "Accurately" requires something to be accurate *about*. So the representation must satisfy these requirements:

#	Requirement	Why it matters
R1	**Measurable**: can be compared with a known answer and scored	The project must *show* the inference is accurate (evaluation plan, final report)
R2	**Predictive**: given a program, says what the student will answer	The agent infers the model from answers; it needs a link from model to answer
R3	**Actionable**: each part maps to a teaching move	The senior-project lesson engine must know *what* to fix, not just *whether* the student is wrong
R4	**Interpretable**: a teacher or student can read it	Trust, debugging, and explaining results in the report and presentation
R5	**Buildable without student data**	No real student data exists yet, and collecting it is a senior-project task (IRB)
R6	**Tractable**: exact inference in milliseconds	The agent updates after every answer, inside an interactive session
R7	**Extensible**: the same structure works for the next lesson	The senior project adds functions in RISC-V (the call stack in assembly)

3. The decision

3.1 What a mental model is in this system

- A **catalog** of misconceptions, each a known wrong version of one notional-machine rule.
- A **profile**: the subset of the catalog a student holds. With 8 entries there are 2^8 = **256 profiles**.
- The **empty profile** stands for a viable model of the notional machine (for this scope).
- A **belief state**: a probability for each of the 256 profiles, owned by the application, never by the LLM.

3.2 The catalog (v1)

Two tiers. The assignment tier comes first because parameter passing in Python *is* assignment, so those rules are prerequisites for the function tier.

Id	Tier	What the student believes	Rule it replaces (§4)
A1_LINKED	Assignment	After `b = a`, the two variables stay linked; reassigning one changes the other	R-assign
A2_REVERSED	Assignment	`x = y` copies the value from left to right	R-assign
A3_COPY_ON_ASSIGN	Assignment	`b = a` (or passing a list) makes an independent copy	R-assign
F1_RETURN_IS_PRINT	Function	`return` and `print` are the same thing	R-return
F2_PARAM_LINKED	Function	Reassigning a parameter changes the caller's variable	R-param
F3_DEF_RUNS_BODY	Function	The body runs when `def` is reached	R-def
F4_LOCALS_OUTLIVE_CALL	Function	Local variables still exist after the call ends	R-frame
F5_RUNS_AFTER_RETURN	Function	Execution continues past `return`	R-return

Each entry stores an id, a tier, a one-sentence statement of the belief, and the typical signal in a student's explanation (used by the LLM interpreter). The first three are documented in the research on value and reference assignment (Ma et al., 2007; 2011); all eight fall within the categories reviewed by Qian and Lehman (2017).

3.3 What the agent outputs

At the end of a session the agent outputs a **diagnosis profile**: the probability of each misconception, the most likely profile, and the session's terminal reason. This is the interface to the senior project's lesson engine.


```
{"marginals": {"A1_LINKED": 0.04, "F1_RETURN_IS_PRINT": 0.93, "F4_LOCALS_OUT
 "diagnosis": ["F1_RETURN_IS_PRINT", "F4_LOCALS_OUTLIVE_CALL"],
 "terminal": "DIAGNOSED", "probes_used": 7}
```

4. Why this counts as a *mental model*, not a list of errors

A notional machine is "the idealised model of the computer implied by the constructs of the programming language" (du Boulay, 1986); Sorva (2013) argues it should be an explicit learning objective. For this scope it has five rules:

Rule	The correct notional machine
R-assign	Assignment binds a name to an object. It never copies and never links two names to each other.
R-def	`def` binds a name to a function. The body does not run until the function is called.
R-param	A call creates a new frame; each parameter is a new local name bound to the argument object.
R-frame	Local names exist only in their frame and disappear when the call returns.
R-return	`return` ends the call immediately and hands a value back to the caller without displaying it.

Every catalog entry replaces exactly one of these rules. A profile therefore describes **the machine the student is actually running in their head**. Because it is a machine, it can be *run*.

Worked example. Probe P11:

```
def setup():
    value = 3
    return value + 1

result = setup()
print(result, value)
```

Student's machine	Predicted answer
Correct notional machine	an error (`value` no longer exists after the call)
Correct, except R-frame replaced by `F4_LOCALS_OUTLIVE_CALL`	`4 3`

If the student answers `4 3`, the profiles containing F4 become more likely and the others less likely. That is the whole inference mechanism: **compare what each candidate machine predicts with what the student says**. A plain list of "errors the student made" cannot do this, because it predicts nothing about the next program.

---

5. Precedent: BUGGY and the "bug library" tradition

This representation is a direct application of one of the best-known ideas in intelligent tutoring.

BUGGY (Brown & Burton, 1978). Brown and Burton studied children's errors in multi-column subtraction. They represented the skill as a network of sub-procedures and modeled each systematic error as the correct network with *buggy variants of individual sub-skills substituted in*, then checked whether that reproduced the child's answers. From its catalog of bugs, BUGGY could also generate diagnostic tests, for example to identify a student who "subtracts the smaller digit in each column from the larger digit regardless of which is on top."

What followed.

- **DEBUGGY** (Burton, 1982) extended BUGGY into an automated diagnostic system.
- **Repair Theory** (Brown & VanLehn, 1980) showed that models of observed bugs can be built by *editing a model of the correct skill*, for example replacing the rule that decides when to borrow with one that always borrows, and then asked *why* those particular bugs arise.
- **Bayesian diagnosis of bugs** (Lee & Corter, 2011) used Bayesian networks to diagnose subtraction bugs from students' test answers, the same probabilistic approach this project's belief state takes. Even their simplest network (bug nodes only) diagnosed bugs correctly at least 85% of the time, and diagnosis became more stable when the network also modeled underlying sub-skills. That second finding supports our assignment tier, which plays the role of prerequisite sub-skills for the function tier.

How this project maps onto BUGGY:

BUGGY (subtraction)	This project (Python function calls)
Correct subtraction procedure	Correct notional machine (the five rules in §4)
A bug: a buggy variant of one sub-skill	A misconception: a wrong version of one rule
A student model: correct procedure with bugs substituted	A profile: correct machine with rules replaced
Diagnostic problems chosen to separate bugs	Probes chosen by expected information gain
Diagnosis: which bugs reproduce the answers	Belief state: probability of each profile given the answers

Lessons we carry over, including the warnings:

- The bug library made diagnosis concrete and testable. That is requirement R1.
- Building the library takes real effort, so the approach suits well-defined procedural domains. A notional machine is exactly such a domain: it is defined by the language's semantics.
- A fixed library restricts the system to errors that are already known (§8).
- Students' bugs can be unstable from one problem to the next, which makes reliable diagnosis hard. Our slip model and probabilistic belief exist to cope with this rather than assume

perfectly consistent students.

6. Options considered

Option	What you get	Why not, for this project
LLM-written narrative ("the student seems to think...")	Rich and flexible	You can't score it against ground truth, and two runs may disagree (fails R1; weak on R2)
Mastery score per skill (knowledge tracing; Corbett & Anderson, 1994)	Simple and well studied	Tells you *whether* the student is wrong, not *what they believe*, and the lessons need the "what" (fails R3)
Concept map or graph	Shows how ideas connect	Hard to elicit, and there's no clear way to check it against an answer (fails R1, R2)
Learned latent vector (embedding of student behavior)	Could find patterns nobody named	Needs lots of real student data, and you can't interpret it (fails R4, R5)
Rule-based catalog of misconceptions (chosen)	Measurable, executable, interpretable, and maps directly to lesson actions	Closed world: only finds known misconceptions (see §8)

Fit against the requirements (✔ meets, ~ partly, ✖ does not):

Requirement	Narrative	Mastery scores	Concept map	Latent vector	Catalog
R1 Measurable	✖	✔	✖	~	✔
R2 Predictive	✖	~ (right or wrong only)	✖	~	✔ (the exact answer)
R3 Actionable	~	✖	~	✖	✔
R4 Interpretable	✔	✔	✔	✖	✔
R5 No student data needed	✔	~ (parameters need fitting)	✔	✖	✔
R6 Tractable	✔	✔	~	✔	✔ (256 profiles, exact)
R7 Extensible to RISC-V	✔	✔	~	✖	✔ (new rules, same structure)

The LLM is not discarded; it is moved to where it is strong. It interprets free-text explanations into catalog labels and, in week 2, chooses which probe to ask. It never holds or edits the belief state.

---

7. How the representation is used across the system

Component	How it uses the catalog
Probe bank (`probes.py`)	Each probe stores the correct output and, for each relevant misconception, the output that misconception predicts
Belief state (`belief.py`)	Exact posterior over all 256 profiles; slip likelihood (a student gives their model's answer with probability 1 – slip)
Probe selection (`select.py`)	Picks the probe whose answer is expected to reduce uncertainty over profiles the most
Evidence interpreter (`interpret.py`)	The LLM maps a free-text explanation onto catalog ids through a forced tool call; unknown ids are rejected
Simulated students (`simstudent.py`)	Ground truth for evaluation: a planted profile plus random slips
Evaluation (`poc.py`)	Scores the diagnosis against the planted profile: exact match, per-misconception precision and recall, Brier score
Lesson engine (senior project)	Reads the diagnosis profile and picks the matching teaching move (for F4, show the frame being popped)

8. Limitations and how we handle them

Limitation	What it means	How we handle it now
Closed world	Only misconceptions already in the catalog can be diagnosed; the same limit applied to BUGGY	Unrecognized answers are classified `OTHER`. A high `OTHER` rate is reported as "unmodeled behavior" rather than forced into a diagnosis
No model vs. wrong model	The empty profile means "no known misconception," which is not the same as having a viable model; a guessing student also gives scattered answers	Reported as a limitation; a "no consistent model" state is planned (§9)
Fragile, context-dependent knowledge	Novices may hold inconsistent ideas that depend on context (diSessa, 1993; Smith, diSessa & Roschelle, 1994); Sorva (2013)	The slip rate absorbs some inconsistency; context-

Limitation	What it means	How we handle it now
	notes students may hold several, even contradictory, models	dependence is a stated limitation
Binary entries	A misconception is present or absent, with no strength or degree	Acceptable for the PoC; probabilities express the agent's uncertainty, not the student's strength of belief
Hand-written catalog and answer keys	Quality depends on the literature and on the author	Grounded in published work; every probe's correct answer is checked against real Python by a test
Evaluation partly circular	Simulated students follow the same answer keys the model assumes, so accuracy is an upper bound	Students use a different slip rate than the model; LLM-played students with free-text answers in week 2; real students in the senior project
Flat list allows contradictory pairs	Nothing stops a profile containing both `A1_LINKED` and `A3_COPY_ON_ASSIGN`, which are competing answers to the same question	Tolerated in v1; fixed by the planned restructuring (§9)

9. Planned work (not in the HW-10 proof of concept)

P1. Organize the catalog by rule, with mutually exclusive alternatives. Each notional-machine rule gets exactly one value per student: the correct version or one of its known wrong versions.

Rule	Student's version (exactly one per rule)
R-assign	correct · linked · copies · reversed
R-def	correct · runs the body
R-param	correct · linked to the caller's variable
R-frame	correct · outlives the call
R-return	correct · same as print · execution continues

This removes contradictory profiles, makes the profile space smaller and more honest (4 × 2 × 2 × 2 × 3 = 96 instead of 256), and sharpens the explanation: *the agent infers which version of each rule the student's machine runs.* It aligns with work that frames program behavior as explicit rules (Duran, Sorva & Seppälä, 2021). Estimated cost: about half a day to rework `belief.py` and the probe keys.

P2. Add a "no consistent model" state per rule for students who are guessing, which addresses the second limitation in §8.

P3. Replace hand-written answer keys with buggy interpreters: one small interpreter per rule variant, in the spirit of BUGGY's executable bug models. Keys are then generated, not authored, and any new probe gets keys for free.

P4. Add the RISC-V call-stack rules for the senior project: `jal` and `ra`, the stack pointer and frame layout, argument and saved registers. Same structure, new rules and variants.

---

10. How to explain this decision

In one sentence: "We represent a student's mental model as the correct notional machine with some of its rules swapped for known wrong versions, and the agent infers which ones by comparing what each version predicts with what the student actually answers."

In one paragraph: "A notional machine is a small set of rules for how code runs. Research has documented the specific wrong versions of those rules that students commonly hold. We put them in a catalog, so a student's mental model is just the set of wrong rules they're using, and no wrong rules means a viable model. Because each version is a set of rules, we can run it: for any program, we know what a student with that model would say it prints. The agent asks short tracing questions, compares the answers with each model's prediction, and updates a probability for every possible model. It's the same idea BUGGY used in 1978 to diagnose children's subtraction bugs, applied to Python function calls."

Questions you are likely to get:

- *Isn't a fixed catalog too limiting?* — Yes, for discovery; it can't find new misconceptions. But it is what makes accuracy measurable, and unrecognized answers are flagged rather than forced into a label. Discovery is a later research question.
- *Why not just let the LLM decide what the student thinks?* — Its judgment can't be scored or reproduced. The LLM still does what it is good at: interpreting free-text explanations into catalog labels and choosing questions, but the application owns the belief.
- *Why Bayesian?* — Students make careless mistakes. A probability over models lets one slip shift the belief a little instead of flipping the diagnosis, and it tells us when the agent is confident enough to stop.
- *How do you know it works?* — Simulated students with planted misconceptions give ground truth. In the first run, the adaptive agent recovered the exact planted profile for 71% of 100 students versus 47% for a fixed-order quiz (`runs/poc-2026-11-03/`).

References

- Brown, J. S., & Burton, R. R. (1978). Diagnostic models for procedural bugs in basic mathematical skills. *Cognitive Science, 2*(2), 155–192. https://doi.org/10.1207/s15516709cog0202_4
- Brown, J. S., & VanLehn, K. (1980). Repair theory: A generative theory of bugs in procedural skills. *Cognitive Science, 4*(4), 379–426.
- Burton, R. R. (1982). Diagnosing bugs in a simple procedural skill. In D. Sleeman & J. S. Brown (Eds.), *Intelligent Tutoring Systems* (pp. 157–183). Academic Press.
- Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction, 4*(4), 253–278.
- diSessa, A. A. (1993). Toward an epistemology of physics. *Cognition and Instruction, 10*(2–3), 105–225.
- du Boulay, B. (1986). Some difficulties of learning to program. *Journal of Educational Computing Research, 2*(1), 57–73.
- Duran, R., Sorva, J., & Seppälä, O. (2021). Rules of program behavior. *ACM Transactions on Computing Education, 21*(4), 1–37.
- Lee, J., & Corter, J. E. (2011). Diagnosis of subtraction bugs using Bayesian networks. *Applied Psychological Measurement, 35*(1), 27–47. https://doi.org/10.1177/0146621610377079
- Ma, L., Ferguson, J., Roper, M., & Wood, M. (2007). Investigating the viability of mental models held by novice programmers. *Proceedings of the 38th SIGCSE Technical Symposium on Computer Science Education*.
- Ma, L., Ferguson, J., Roper, M., & Wood, M. (2011). Investigating and improving the models of programming concepts held by novice programmers. *Computer Science Education, 21*(1), 57–80. https://doi.org/10.1080/08993408.2011.554722
- Qian, Y., & Lehman, J. (2017). Students' misconceptions and other difficulties in introductory programming: A literature review. *ACM Transactions on Computing Education, 18*(1), 1–24.
- Smith, J. P., diSessa, A. A., & Roschelle, J. (1994). Misconceptions reconceived: A constructivist analysis of knowledge in transition. *Journal of the Learning Sciences, 3*(2), 115–163.
- Sorva, J. (2013). Notional machines and introductory programming education. *ACM Transactions on Computing Education, 13*(2), 8:1–8:31. https://doi.org/10.1145/2483710.2483713